

BLACKBOX X — PHASE 4.1

RESEARCH REPRODUCIBILITY & DECISION AUDIT LAYER
ARCHITECTURE — Formal Case Encapsulation, Manifest
Specification, Multi-Level Replay Verification, Evidence Lineage,
and Research Diff Engine — 1.1 Objective

DOCUMENT
**RESEARCH-AUDIT-
ARCHITECTURE.pdf**

SYSTEM
BLACKBOX X Engine

SOURCE
**docs/RESEARCH-AUDIT-
ARCHITECTURE.md**

STATUS
Documentation

Table of Contents

RESEARCH REPRODUCIBILITY & DECISION AUDIT LAYER ARCHITECTURE

Formal Case Encapsulation, Manifest Specification, Multi-Level Replay Verification, Evidence Lineage, and Research Diff Engine

1. Executive Summary & Purpose

1.1 Objective

1.2 Core Architectural Invariant: Zero Financial Calculation Duplication

2. Existing Architecture Reuse Map

3. Research Case Data Contract

4. Case Identity Architecture & Hierarchy

4.1 Human-Readable Identifier Generation

5. Machine-Readable Research Manifest Contract

6. Deterministic Replay Engine

6.1 Replay State Transitions

7. The Mathematical Reproducibility Contract

7.1 Scope of the Reproducibility Guarantee

7.2 Numerical Formatting & Floating-Point Normalization

8. Six-Level Replay Verification Algorithm

8.1 ReplayVerificationRecord Contract

9. Neutral Research Diff Engine

10. Evidence Lineage Model

11. Claim Inspector Component Contract

12. Auditable Case File Structure (17 Sections)

13. Audit Timeline Specification

14. Version Compatibility Model

15. Security & Threat Model

16. Performance & Memory Ceilings

17. Ghost Mode & Research Trail Integration

17.1 Ghost Mode 2.0 Reproducibility Events

17.2 Research Trail Permanent Journaling

18. AI Boundaries & Epistemic Protocol

19. Institutional UI Architecture: Research Audit Console

20. Comprehensive Testing Strategy (35 Automated Invariants)

21. Failure Modes & Graceful Handling Matrix

22. Proposed Implementation File Architecture

23. Acceptance Criteria Checklist

BLACKBOX X — PHASE 4.1

RESEARCH REPRODUCIBILITY & DECISION AUDIT LAYER ARCHITECTURE

Formal Case Encapsulation, Manifest Specification, Multi-Level Replay Verification, Evidence Lineage, and Research Diff Engine

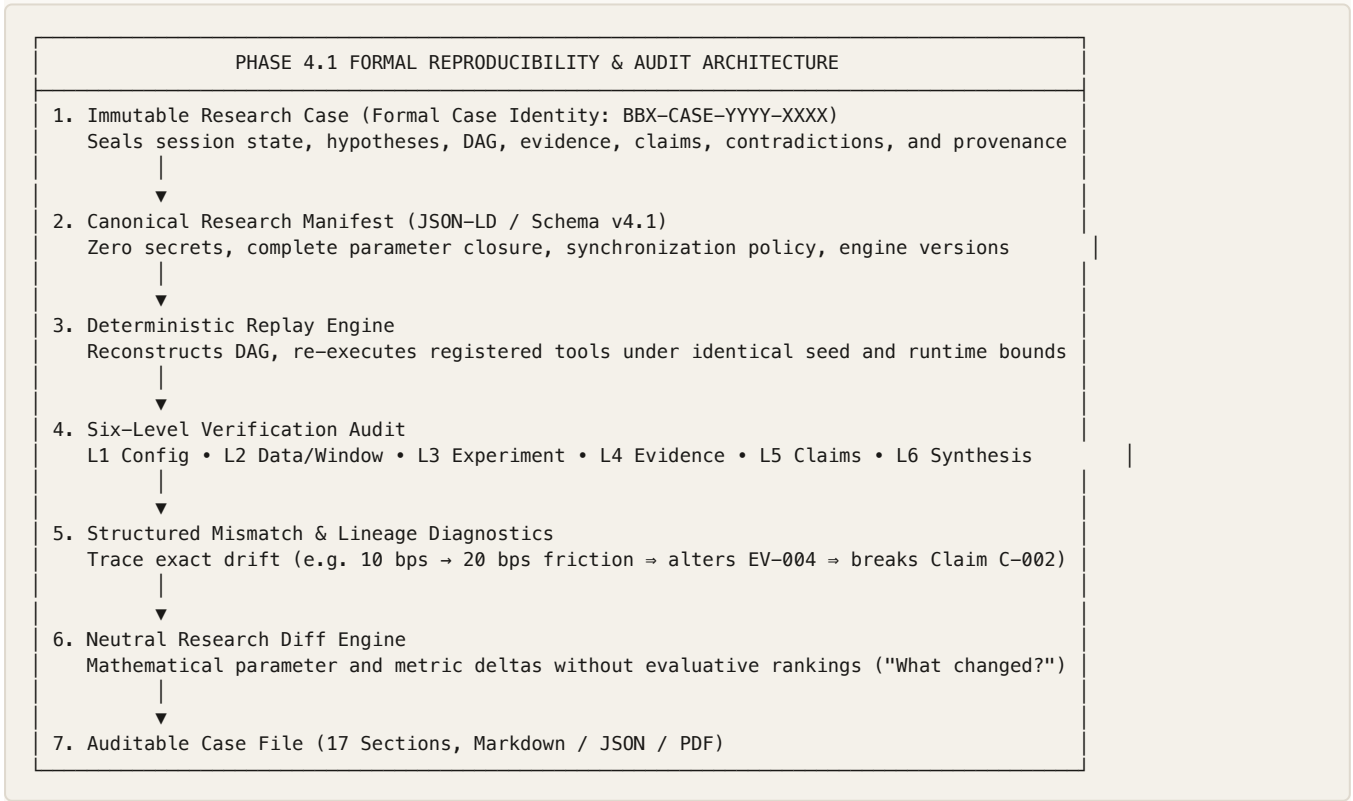
1. Executive Summary & Purpose

1.1 Objective

Phase 4.1 establishes a formal, tamper-evident **Reproducibility, Replay, and Decision Audit Layer** on top of the Phase 4.0 Institutional Research Workspace.

Phase 4.0 transformed BLACKBOX X into an orchestrated research workspace producing falsifiable hypotheses, bounded experiment DAGs, direct evidence records, contradiction evaluations, secondary sensitivity sweeps, structured numerical claims, and reproducible research memos. Phase 4.1 elevates completed sessions into **immutable Research Cases**, encapsulates complete reproducible configurations into machine-readable **Research Manifests**, provides a deterministic **Replay Engine** with 6-level verification, renders an interactive **Evidence Lineage** graph, and delivers a neutral **Research Diff Engine** for auditing parameter drift and evidence divergence across research investigations:

Research Inquiry → Research Case → Research Manifest → Evidence Lineage
→ Replay Engine → 6-Level Verification → Structured Diff → Auditable Case File



1.2 Core Architectural Invariant: Zero Financial Calculation Duplication

Phase 4.1 is strictly an **audit, reproducibility, lineage, and verification layer**.

- **No Calculation Duplication:** Phase 4.1 implements zero financial metric calculations, backtest loops, return calculations, covariance estimations, Monte Carlo random-number generators, or regime detection algorithms.
- **Single Source of Financial Truth:** Quantitative engines (Phases 1 through 3.9) remain the sole source of truth. When replaying experiments, the Replay Engine dispatches identical inputs exclusively to registered tools via `ResearchToolRegistry`.
- **AI Boundaries:** Large Language Models operate strictly as explainers and summarizers of audit timelines, lineages, and differences. AI **never** alters manifests, modifies evidence records, tampers with fingerprints, or fabricates numerical claim values.

2. Existing Architecture Reuse Map

Phase 4.1 integrates with and strictly preserves all locked components from Phases 1 through 4.0:

ARCHITECTURE REUSE MAPPING	
Component	Phase 4.1 Integration & Reuse Modality
Quantitative Metrics	Replay validation target via <code>`get_asset_metrics`</code>
Strategy Engine & Backtest	Replay execution via <code>`get_strategy_metrics`</code> & benchmark
Correlation Engine	Matrix & rolling verification via <code>`get_correlation_*`</code>
Market Regimes	4-regime classification via <code>`get_regime_performance`</code>
Stress Testing (Phase 3.2)	Macro crisis shock replay via <code>`get_stress_result`</code>
Strategy Genome (Phase 3.3)	5D risk fingerprint replay via <code>`get_strategy_genome`</code>
Risk Committee (Phase 3.4)	ResearchPack audit export and CRO verification briefing
AI Assistant (Phase 3.5)	Lineage explainer & audit timeline narrative
Quant Research Agent (3.6)	Falsifiable hypothesis & contradiction engine patterns
Portfolio Intelligence (3.7)	Simplex allocations & Euler risk replay verification
Monte Carlo (Phase 3.8)	Unconditional bootstrap/parametric replay (Web Worker)
Regime Monte Carlo (3.9)	Markov transition simulation replay (Mulberry32 seed)
ResearchSession (Phase 4.0)	Underlying operational session captured by ResearchCase
ResearchToolRegistry (4.0)	Exclusive execution allowlist for deterministic replay
EvidenceRecord & Claims (4.0)	Atomic objects bound into lineage graphs and verified
researchFingerprint (4.0)	Canonical JSON serialization & SHA-256 hash algorithms

3. Research Case Data Contract

An active `ResearchSession` (Phase 4.0) transitions upon completion into an immutable `ResearchCase`. Once locked, a `ResearchCase` cannot be mutated.

```

export type CaseStatus =
| 'SEALED' // Normal completed session, permanently locked
| 'REPLAY_VERIFIED' // Successfully replayed with 100% bit-equivalent match
| 'REPLAY_DIVERGENT' // Replay exhibited parameter or evidence divergence
| 'ARCHIVED' // Historical reference case
| 'REVOKED'; // Invalidated by audit due to corrupted input data

export interface ResearchCase {
  caseId: string; // Canonical identity: "BBX-CASE-2026-XXXX"
  caseVersion: number; // Incremental case schema version (e.g., 1)
  createdAt: number; // Epoch timestamp ms of case creation
  sealedAt: number; // Epoch timestamp ms of permanent sealing
  updatedAt: number; // Epoch timestamp ms of latest audit action
  sessionId: string; // Originating ResearchSession UUID
  question: string; // Raw natural language question investigated
  normalizedQuestion: string; // Sanitized, lowercased canonical inquiry
  status: CaseStatus;

  // Bound Research Entity References
  manifestId: string; // Bound ResearchManifest identifier
  hypothesisIds: string[]; // Ordered list of evaluated hypotheses
  experimentIds: string[]; // Ordered list of planned experiment DAG nodes
  evidenceIds: string[]; // Ordered list of generated EvidenceRecords
  claimIds: string[]; // Structured ResearchClaim identifiers
  contradictionIds: string[]; // Resolved contradiction record IDs
  secondaryTestIds: string[]; // Triggered secondary sensitivity sweep IDs
  synthesisId: string; // ResearchSynthesis identifier
  memoId: string; // Generated ResearchMemo reference

  // Audit & Reproducibility Payloads
  manifest: ResearchManifest; // Embedded standalone reproducible manifest
  provenance: CaseProvenance; // Complete multi-engine data & runtime context
  reproducibility: ReproducibilityContract; // Formal mathematical reproducibility contract
  lineage: CaseLineage; // Forward and backward claim-to-evidence graph
  auditTimeline: AuditEvent[]; // Tamper-evident append-only event trail
  verificationHistory: ReplayVerificationRecord[]; // Log of all replay executions
}

export interface CaseProvenance {
  datasetWindow: {
    startDate: string; // "2019-01-01"
    endDate: string; // "2023-12-31"
    observationCount: number; // Exactly 1,825 bars
    synchronizationPolicy: string; // Common trading calendar intersection
    datasetFingerprint: string; // "bx-sync-1825-gold-btc-nvda-v1"
  };
  assetUniverse: Array<'GOLD' | 'BTC' | 'NVDA'>;
  engineVersions: {
    quantCore: string; // "1.0"
    regimeEngine: string; // "2.0"
    stressEngine: string; // "3.2"
    genomeEngine: string; // "3.3"
    riskCommittee: string; // "3.4"
    portfolioEngine: string; // "3.7"
    monteCarloEngine: string; // "3.8"
    regimeMonteCarloEngine: string; // "3.9"
    workspaceEngine: string; // "4.0"
    auditEngine: string; // "4.1"
  };
  runtimeEnvironment: {
    nodeVersion?: string;
    browserRuntime?: string;
    numericalContract: 'IEEE-754-FLOAT64-ROUND-NEAREST-TIES-EVEN';
  };
  deterministicSeed: number; // PRNG seed (e.g. 42 or 100)
  totalToolsExecuted: number;
  totalExecutionDurationMs: number;
}

```

4. Case Identity Architecture & Hierarchy

To eliminate ambiguity between human identifiers, cryptographic hashes, and runtime instances, Phase 4.1 establishes a four-tier identity architecture:

FOUR-TIER IDENTITY ARCHITECTURE		
Identifier Tier	Format / Scheme	Purpose & Scope
1. Human-Readable Case Identifier (caseId)	`BBX-CASE-YYYY-[SEQ4]` E.g., `BBX-CASE-2026-0042`	Human citation, filing, editorial memos, Risk Committee
2. Canonical Session Fingerprint	`SHA-256(Q    D    E    H    S)` E.g., `bx-8f2c3a91b4e07d15`	Cryptographic fingerprint of total configuration parameters
3. Experiment Fingerprint	`SHA-256(Tool    Args    Win)` E.g., `bx-exp-4c19a02e88f1`	Idempotency caching and node equivalence during replay runs
4. Evidence Fingerprint	`SHA-256(Exp    Result)` E.g., `bx-ev-99b1e4c70312`	Bit-level numerical payload integrity & lineage tracking

4.1 Human-Readable Identifier Generation

The `caseId` follows a strict sequential standard: `BBX-CASE-{YEAR}-{SEQUENCE_4_DIGITS}`.

- Deterministically allocated by the case store.
- Independent of arbitrary runtime UUIDs.
- Guaranteed unique within the workspace repository.

5. Machine-Readable Research Manifest Contract

The `ResearchManifest` is a self-contained, machine-readable declarative document capturing every parameter required to reproduce the investigation bit-for-bit:

```

export interface ResearchManifest {
  manifestVersion: '4.1.0';
  manifestId: string; // "MNF-BBX-CASE-2026-0042"
  createdAt: string; // ISO 8601 UTC string
  caseId: string;

  // 1. DATA MANIFEST
  data: {
    assetUniverse: Array<'GOLD' | 'BTC' | 'NVDA'>;
    assetCanonicalOrdering: ['GOLD', 'BTC', 'NVDA'];
    startDate: string; // "2019-01-01"
    endDate: string; // "2023-12-31"
    observationCount: 1825;
    synchronizationPolicy: 'CONTEMPORANEOUS_COMMON_CALENDAR_INTERSECTION';
    datasetFingerprint: string; // "bx-sync-1825-gold-btc-nvda-v1"
  };

  // 2. STRATEGY & BACKTEST MANIFEST
  strategy: {
    targetAsset: 'GOLD' | 'BTC' | 'NVDA';
    targetStrategy: 'SMA_CROSSOVER' | 'EMA_TREND' | 'MOMENTUM' | 'MEAN_REVERSION';
    strategyParameters: Record<string, number | string>;
    executionConvention: 'NEXT_BAR_CLOSE_WITH_ZERO_LOOKAHEAD';
    transactionCostBps: number; // E.g., 10 (0.10%)
    initialCapital: number; // 100,000
    benchmarkAsset: 'GOLD' | 'BTC' | 'NVDA';
  };

  // 3. PORTFOLIO CONFIGURATION MANIFEST
  portfolio?: {
    weights: Record<'GOLD' | 'BTC' | 'NVDA', number>;
    rebalanceSchedule: 'DAILY' | 'MONTHLY' | 'QUARTERLY' | 'BUY_AND_HOLD';
    riskFreeRateAnnPct: 4.0;
    covarianceMethod: 'SAMPLE_ANNUALIZED_252';
  };

  // 4. PROBABILISTIC SIMULATION MANIFEST
  simulation?: {
    monteCarloMethod: 'STATIONARY_CIRCULAR_BOOTSTRAP' | 'STUDENT_T_PARAMETRIC' | 'GAUSSIAN_PARAMETRIC';
    regimeConditioned: boolean;
    startingRegime?: 'BULL' | 'BEAR' | 'HIGH_VOLATILITY' | 'LOW_VOLATILITY' | 'EMPIRICAL_DISTRIBUTION';
    simulationCount: number; // Ceiling: 10,000
    horizonDays: number; // E.g., 252
    deterministicSeed: number;
    prngAlgorithm: 'MULBERRY_32';
  };

  // 5. ENGINE STACK MANIFEST
  engines: Record<string, string>;

  // 6. RESEARCH WORKFLOW MANIFEST
  research: {
    researchQuestion: string;
    hypotheses: Array<{
      id: string;
      category: string;
      statement: string;
      testIds: string[];
    }>;
    experimentDAG: Array<{
      experimentId: string;
      toolName: string;
      arguments: Record<string, any>;
      dependencies: string[];
      seed?: number;
      fingerprint: string;
    }>;
    expectedEvidenceFingerprints: Record<string, string>; // expId -> expected SHA-256
    expectedClaims: Array<{
      claimId: string;
      metric: string;
      value: number | string;
    }>;
  };
}

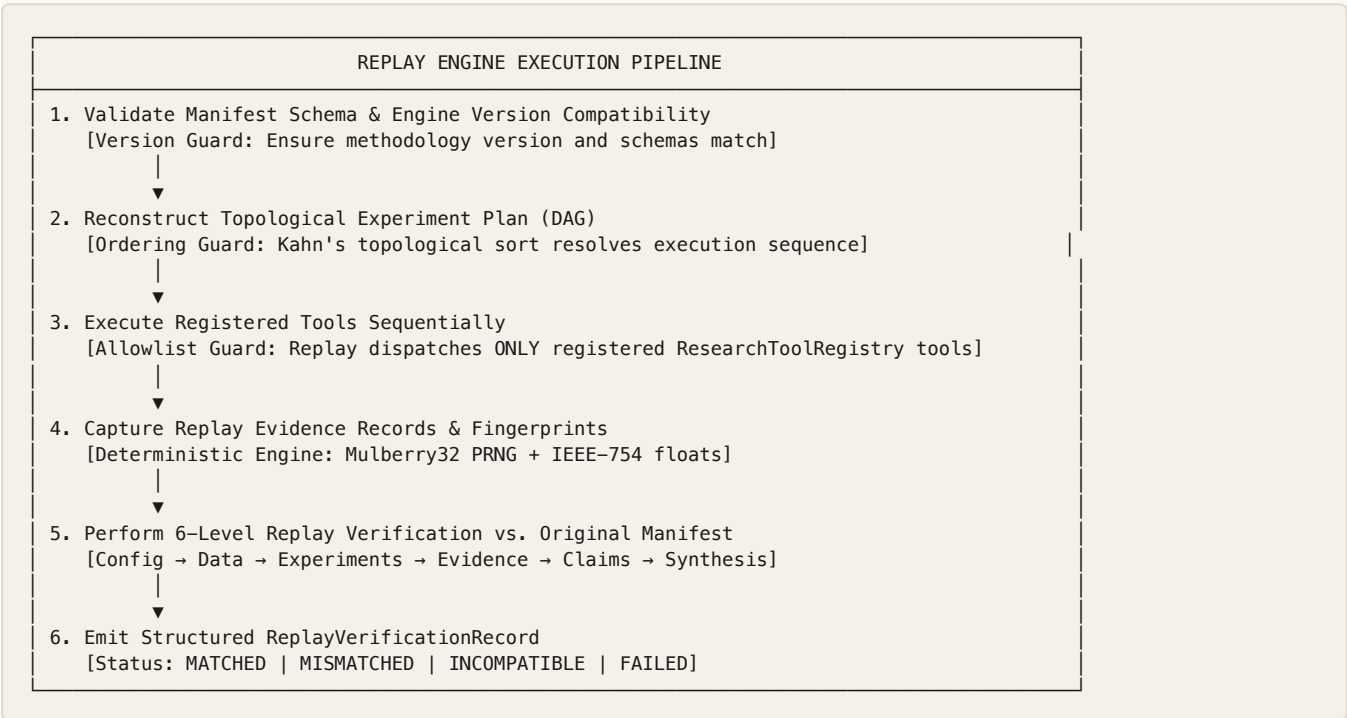
```

```
        boundEvidence: string[];
    }>;
};

// 7. SECURITY & INTEGRITY CERTIFICATE
security: {
    containsZeroApiKeys: true;
    containsZeroExecutableCode: true;
    isSanitizedAgainstInjection: true;
    canonicalManifestHash: string;           // SHA-256 of canonical JSON
};
}
```

6. Deterministic Replay Engine

The `ResearchReplayEngine` consumes a `ResearchCase` or `ResearchManifest` and systematically re-executes the planned quantitative experiments to verify whether outputs match the original investigation.



6.1 Replay State Transitions

```
export type ReplayState =
  | 'PENDING'
  | 'VALIDATING_MANIFEST'
  | 'REPLAYING_EXPERIMENTS'
  | 'VERIFYING_EVIDENCE'
  | 'VERIFYING_CLAIMS'
  | 'MATCHED'
  | 'MISMATCHED'
  | 'INCOMPATIBLE'
  | 'FAILED';
```

7. The Mathematical Reproducibility Contract

Universal bit-level identity cannot be promised across unconstrained runtimes (different compilers, non-standard WASM runtimes, or altered SIMD rounding). Therefore, Phase 4.1 defines the **BLACKBOX X Reproducibility Contract**:

7.1 Scope of the Reproducibility Guarantee

SAME ENGINE VERSIONS + SAME CALIBRATED DATASET

+ SAME PARAMETERS

+ SAME SEED

+ IEEE-754 FLOAT64 RUNTIME

⇒ BIT-IDENTICAL OUTPUT FINGERPRINTS

7.2 Numerical Formatting & Floating-Point Normalization

1. **Canonical Floats:** Floating-point numbers with decimal fractions are formatted to 8 decimal places (`toFixed(8)`) during serialization to eliminate cross-platform stringification variances.
2. **Special Values:** `NaN` serialized strictly as `"NaN"`; `+Infinity` as `"Infinity"`; `-Infinity` as `"-Infinity"`.
3. **Simplex Sums:** Simplex portfolio weights are enforced to $\sum w_i = 1.0$ within $\varepsilon = 10^{-6}$.
4. **Deterministic PRNG:** All stochastic engines (Phase 3.8 / 3.9) utilize Mulberry32 with explicit seeds. `Math.random()` is strictly forbidden.
5. **Key Sorting:** All JSON objects are canonicalized recursively by sorting keys in lexicographical Unicode order prior to SHA-256 hashing.

8. Six-Level Replay Verification Algorithm

Replay verification does not simply emit a boolean flag; it tests six hierarchical evidentiary tiers and generates structured mismatch diagnostics:

SIX-LEVEL VERIFICATION AUDIT		
Level	Verification Tier	Evaluation Criteria
Level 1	Configuration Audit	Asset universe, strategy params, cost bps, seeds, weights
Level 2	Data Window Audit	Start/End date, synchronized observation count (1,825)
Level 3	Experiment Plan Audit	Planned DAG nodes, dependency order, tool allowlist
Level 4	Evidence Audit	Numerical results match bit-for-bit (tolerance: 1e-6)
Level 5	Research Claim Audit	Structured claims bound to identical Evidence IDs
Level 6	Synthesis Audit	Findings, contradiction outcomes, and causality flags

8.1 ReplayVerificationRecord Contract

```
export interface ReplayVerificationRecord {
  verificationId: string; // "VRF-2026-0042-01"
  caseId: string;
  replayedAt: number;
  overallStatus: 'MATCHED' | 'MISMATCHED' | 'INCOMPATIBLE' | 'FAILED';

  tierResults: {
    level1Configuration: { status: 'PASS' | 'FAIL'; mismatches: string[] };
    level2DataWindow: { status: 'PASS' | 'FAIL'; mismatches: string[] };
    level3ExperimentPlan: { status: 'PASS' | 'FAIL'; mismatches: string[] };
    level4EvidenceRecords: { status: 'PASS' | 'FAIL'; mismatches: EvidenceMismatch[] };
    level5ResearchClaims: { status: 'PASS' | 'FAIL'; mismatches: ClaimMismatch[] };
    level6Synthesis: { status: 'PASS' | 'FAIL'; mismatches: string[] };
  };

  fingerprintComparison: {
    originalFingerprint: string;
    replayFingerprint: string;
    identical: boolean;
  };

  totalReplayLatencyMs: number;
}

export interface EvidenceMismatch {
  evidenceId: string;
  toolName: string;
  metric: string;
  originalValue: number | string;
  replayValue: number | string;
  absoluteDelta: number;
  relativeDeltaPct?: number;
}

export interface ClaimMismatch {
  claimId: string;
  metric: string;
  originalClaimText: string;
  replayedEvidenceValue: number | string;
  reason: string;
}
```

9. Neutral Research Diff Engine

The `ResearchDiffEngine` provides multi-case comparative auditing between two `ResearchCases` (*A* and *B*).

- **Strict Epistemic Neutrality Protocol:** The Diff Engine strictly forbids subjective evaluations, performance scores, or rankings (e.g., "*Case A outperformed Case B*" or "*Case A is superior*").
- **Objective Delta Output:** Reports strictly what changed, absolute deltas, relative percentage shifts, and direction.

```

export interface ResearchDiffResult {
  caseAId: string;
  caseBId: string;
  comparedAt: number;

  inquiryDiff: {
    identicalQuestion: boolean;
    questionA: string;
    questionB: string;
  };

  parameterDiffs: Array<{
    category: 'DATA' | 'STRATEGY' | 'COST' | 'PORTFOLIO' | 'SIMULATION';
    parameterName: string;
    valueA: any;
    valueB: any;
    isDivergent: boolean;
  }>;

  quantitativeMetricDiffs: Array<{
    metricName: string;
    category: 'RETURN' | 'VOLATILITY' | 'SHARPE' | 'DRAWDOWN' | 'VAR' | 'TURNOVER';
    valueA: number;
    valueB: number;
    absoluteDelta: number;
    relativeDeltaPct?: number;
    direction: 'INCREASE' | 'DECREASE' | 'NO_CHANGE';
    evidenceRefA: string;
    evidenceRefB: string;
  }>;

  hypothesisDiffs: Array<{
    hypothesisId: string;
    statement: string;
    statusA: string;
    statusB: string;
    confidenceA: string;
    confidenceB: string;
    statusShift: boolean;
  }>;

  contradictionDiffs: {
    contradictionsA: number;
    contradictionsB: number;
    summary: string;
  };

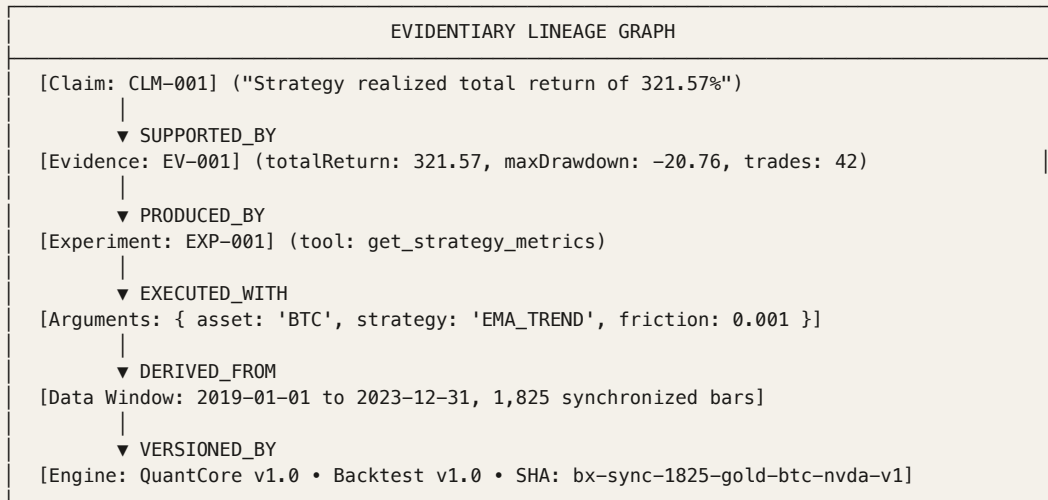
  reproducibilityDiff: {
    seedA: number;
    seedB: number;
    fingerprintA: string;
    fingerprintB: string;
    identicalFingerprint: boolean;
  };
}

```

10. Evidence Lineage Model

Every `ResearchClaim` in a research memo must be bidirectionally traceable down to its underlying data bar:

Claim → Evidence Record → Experiment Node → Tool Definition → Arguments → Data Window → Engi



```
export interface LineageTrace {
  claimId: string;
  claimText: string;
  metric: string;
  claimedValue: number | string;
  evidence: {
    evidenceId: string;
    directEvidence: string;
    interpretation: string;
    evidenceFingerprint: string;
  };
  experiment: {
    experimentId: string;
    toolName: string;
    arguments: Record<string, any>;
    seed?: number;
    latencyMs?: number;
    experimentFingerprint: string;
  };
  provenance: {
    startDate: string;
    endDate: string;
    observationCount: number;
    datasetFingerprint: string;
    engineVersion: string;
  };
}
```

11. Claim Inspector Component Contract

When the researcher clicks any numerical figure in the Research Memo or Case View, the system opens the **Claim Inspector**:

- **Claim Overview:** Claim ID, metric name, declared value, unit.
- **Direct Evidence:** Empirical factual statement extracted directly from engine output.
- **Interpretation:** Analytical meaning deduced from the direct evidence.
- **Underlying Engine Run:** Canonical tool name (`get_strategy_metrics`), exact input arguments, execution latency, and deterministic seed.
- **Data Window & Dataset Fingerprint:** 1,825 bars (2019–2023), synchronous calendar hash.
- **Cryptographic Hash:** Output fingerprint (`bx-ev-xxxxxxx`).
- **Separation Verification:** Visual alert confirming that Direct Evidence and Interpretation are distinct.

12. Auditable Case File Structure (17 Sections)

The complete `ResearchCaseFile` represents the primary institutional deliverable:

```
=====
BLACKBOX X – AUDITABLE RESEARCH CASE FILE
Case ID: BBX-CASE-2026-0042
=====

1. CASE IDENTITY & CLASSIFICATION
2. RESEARCH QUESTION & SCOPE
3. COMPLETE REPRODUCIBILITY MANIFEST
4. DATASET PROVENANCE & CALENDAR SYNCHRONIZATION AUDIT
5. FALSIFIABLE HYPOTHESES & CONFIDENCE TIERS
6. BOUNDED EXPERIMENT DAG SPECIFICATION
7. IMMUTABLE EXECUTION TIMELINE
8. EVIDENCE GRAPH (TOPOLOGICAL DAG)
9. DIRECT QUANTITATIVE EVIDENCE CATALOG
10. ADVERSARIAL CONTRADICTION & COUNTERFACTUAL AUDIT
11. SECONDARY PARAMETER SENSITIVITY SWEEPS
12. STRUCTURED RESEARCH CLAIMS & LINEAGE MAP
13. REPRODUCIBILITY CONTRACT & FINGERPRINTS
14. REPLAY VERIFICATION AUDIT REPORT
15. NEUTRAL RESEARCH SYNTHESIS
16. METHODOLOGICAL LIMITATIONS & BOUNDARIES
17. NON-INVESTMENT ADVICE & REGULATORY DISCLAIMER
=====
```

13. Audit Timeline Specification

Every event occurring throughout the research and audit lifecycle is captured in an append-only, tamper-evident timeline:

```
export interface AuditEvent {
  eventId: string; // "AUD-001"
  timestamp: number; // Epoch timestamp ms
  isoTimestamp: string; // "2026-09-19T13:42:01.124Z"
  eventType:
    | 'CASE_INITIALIZED'
    | 'HYPOTHESIS_FORMULATED'
    | 'EXPERIMENT_PLAN_LOCKED'
    | 'TOOL_DISPATCHED'
    | 'EVIDENCE_RECORDED'
    | 'CONTRADICTION_FLAGGED'
    | 'SECONDARY_TEST_TRIGGERED'
    | 'SYNTHESIS_COMPILED'
    | 'CASE_SEALED'
    | 'REPLAY_DISPATCHED'
    | 'REPLAY_VERIFIED'
    | 'REPLAY_MISMATCH_DETECTED'
    | 'CASE_EXPORTED';
  actor: 'SYSTEM_ORCHESTRATOR' | 'RESEARCH_REPLAY_ENGINE' | 'ANALYST';
  payload: Record<string, any>;
  eventDigest: string; // SHA-256 of canonical event state
}
```

14. Version Compatibility Model

Replay requests evaluate version compatibility across three levels:

```
export type VersionCompatibilityStatus =
  | 'EXACT_COMPATIBLE'           // Engine and methodology versions match exactly
  | 'COMPATIBLE_WITH_WARNING'   // Minor version bump with backward-compatible numerical contract
  | 'INCOMPATIBLE';             // Major version bump or breaking numerical changes

export function checkVersionCompatibility(
  manifestVersions: Record<string, string>,
  currentVersions: Record<string, string>
): {
  status: VersionCompatibilityStatus;
  divergences: Array<{ engine: string; manifestVer: string; currentVer: string; impact: string }>;
};
```

If an engine has undergone a breaking numerical change (e.g. Monte Carlo algorithm shift from Gaussian to Student-*t*), the Replay Engine blocks execution with `INCOMPATIBLE_ENGINE_VERSION` rather than producing corrupted verification verdicts.

15. Security & Threat Model

AUDIT SECURITY & THREAT DEFENSE	
Threat Vector	Phase 4.1 Mitigation & Defense Guardrail
Secret / API Key Leakage	Strict sanitizer prevents persistence of tokens or keys.
Arbitrary Code Injection	Replay engine executes ONLY registered allowlist tools. <code>eval()</code> and <code>new Function()</code> are strictly prohibited.
Prompt Injection Attack	Untrusted prompt overrides cannot alter manifest hashes, execution budgets, or tool allowlists.
Evidence Forgery	Numerical claims require verified <code>EvidenceRecord</code> binds. Mismatched evidence hashes trigger <code>REPLAY_MISMATCH</code> .
Path / Memory Explosion	Replay simulation paths strictly capped at 10,000. Raw trajectory cubes are never persisted in state.

16. Performance & Memory Ceilings

- Lightweight State Footprint:** A sealed `ResearchCase` contains summary statistics, percentiles, and fingerprints. Raw Monte Carlo paths ($10,000 \times 252 \times 3$ floats $\approx$ 60 MB) are **never** serialized into the manifest or case store.
- Worker Parity:** Heavy simulation tools (`get_monte_carlo_risk`, `get_regime_monte_carlo_risk`) execute in background Web Workers during replay without locking the UI.
- Execution Timeout:** Replay runs inherit the strict 60-second execution ceiling.

17. Ghost Mode & Research Trail Integration

17.1 Ghost Mode 2.0 Reproducibility Events

Ghost Mode monitors replay audits and emits three deterministic insights:

- `REPRODUCIBILITY_STABLE` : Replay reproduced the original evidence fingerprint with 100% bit-equality.
- `PARAMETER_DRIFT` : Two otherwise related cases differ in transaction-cost friction or lookback parameters.
- `EVIDENCE_DRIFT` : Re-running across an altered data window alters a previously observed regime return relationship.

17.2 Research Trail Permanent Journaling

Every sealed `ResearchCase` appends an immutable record to the Research Trail:

- `caseId`
- `question`
- `manifestId`
- `reproducibilityStatus` (`SEALED` / `REPLAY_VERIFIED`)
- `sessionFingerprint`
- `executiveSummary`

18. AI Boundaries & Epistemic Protocol

Permitted AI Functions	Strictly Prohibited AI Functions
Explain parameter differences in a Research Diff	Alter ResearchManifest parameters or hashes
Summarize audit timeline events in clear narrative	Modify EvidenceRecords or forged claim values
Clarify the mathematical meaning of evidence lineage	Override ReplayEngine verification status
Suggest follow-up research questions based on drift	Generate predictive return forecasts or guarantees
Explain why two cases diverged under higher friction	Provide financial, investment, or trading advice

19. Institutional UI Architecture: Research Audit Console

Directory: `src/components/workspace/research/audit/`

```
src/components/workspace/research/audit/
├─ ResearchCaseView.tsx      // Master container for sealed cases
├─ ResearchCaseFile.tsx     // 17-section full case viewer (Markdown / JSON)
├─ ReplayPanel.tsx          // Replay runner with live step-by-step progress
├─ ReplayVerification.tsx   // 6-level verification results table & mismatch drilldown
├─ ResearchDiffView.tsx     // Side-by-side neutral comparison view
├─ EvidenceLineage.tsx      // Interactive claim-to-evidence graph explorer
├─ ClaimInspector.tsx       // Modal drawer for inspecting claim evidence binds
├─ AuditTimeline.tsx        // Append-only chronological event ledger
├─ CaseExportPanel.tsx      // Multi-format export (JSON / Markdown / Printable PDF)
```

INSTITUTIONAL AUDIT CONSOLE UX

[CASE: BBX-CASE-2026-0042] • STATUS: [REPLAY VERIFIED ✓] • AUDIT: [bx-8f2c3a91b4e07d15] |

Tabs: [CASE FILE] [REPLAY AUDIT] [EVIDENCE LINEAGE] [RESEARCH DIFF] [TIMELINE] |

6-LEVEL REPLAY VERIFICATION:

✓ Level 1: Configuration Identical (18/18 params)

✓ Level 2: Data Window Matched (1,825 bars, 2019-01-01 → 2023-12-31)

✓ Level 3: Experiment DAG Order Verified (5 nodes)

✓ Level 4: Evidence Payloads Bit-Identical (EV-001 → EV-005)

✓ Level 5: Numerical Claims Bound & Validated (CLM-001 → CLM-004)

✓ Level 6: Synthesis & Contradiction Resolution Invariant

20. Comprehensive Testing Strategy (35 Automated Invariants)

The verification suite (`tests/researchAudit.test.ts`) will enforce 35 critical audit invariants:

1. **Case Creation:** Sealed case receives valid `BBX-CASE-YYYY-XXXX` identity and timestamps.
2. **Case Immutability:** Modifying properties of a sealed `ResearchCase` is prohibited / throws error.
3. **Manifest Completeness:** Manifest captures all 7 mandated sections (Data, Strategy, Portfolio, Simulation, Engines, Research, Security).
4. **Fingerprint Determinism:** Identical manifest generates identical SHA-256 canonical hash.
5. **Replay Match:** Replaying a case with identical configuration produces `MATCHED` verification.
6. **Replay Mismatch:** Modifying transaction cost from 10 bps to 20 bps produces `MISMATCHED` with structured delta diagnostics.
7. **Version Compatibility:** Exact engine versions produce `EXACT_COMPATIBLE`.
8. **Incompatible Version Rejection:** Replaying against mismatched major engine version yields `INCOMPATIBLE`.
9. **Data Window Validation:** Altering dataset observation count flags Level 2 verification failure.
10. **Parameter Difference Detection:** Diff engine detects single parameter changes across cases.
11. **Evidence Lineage Backward Trace:** Claim traces cleanly back through evidence, experiment, tool, and data window.
12. **Evidence Lineage Forward Trace:** Tool experiment traces forward to bound claims and synthesis statements.
13. **Claim Binding Integrity:** Every numeric claim in case file maps to an existing `EvidenceRecord`.
14. **Unbound Claim Rejection:** Claim lacking evidence link fails case creation.
15. **Audit Timeline Append-Only:** Timeline events are monotonically ordered and immutable.
16. **Diff Engine Neutrality:** Diff report contains zero evaluative adjectives ("*better*", "*worse*", "*superior*").
17. **Quantitative Delta Calculation:** Computes exact absolute and relative deltas for metrics.
18. **Seed Preservation in Replay:** PRNG seed is preserved and passed to replayed simulation tools.
19. **Monte Carlo Replay Parity:** Unconditional Monte Carlo replay reproduces identical percentiles.
20. **Regime Monte Carlo Replay Parity:** Regime Monte Carlo replay reproduces identical transition matrices and MDD percentiles.
21. **Portfolio Simplex Replay:** Portfolio optimization replay reproduces identical asset weight allocations.
22. **Stress Shock Replay:** Stress testing replay reproduces identical stressed drawdowns.
23. **Prompt Injection Sanitization:** Injected prompt strings in research questions fail to alter manifest policies.
24. **Secret Exclusion Audit:** Verified zero API keys, secrets, or bearer tokens in exported manifests.
25. **AI Role Boundary:** AI functions cannot mutate case state or forge evidence.
26. **Worker Sandbox Isolation:** Heavy simulations execute off main thread without memory leaks.
27. **Bounded Memory Footprint:** Large Monte Carlo trajectory matrices are not serialized in case state.
28. **Idempotency Verification:** Successive replays produce identical execution counts and fingerprints.
29. **Corrupted Manifest Recovery:** Malformed JSON manifest halts replay with structured schema error.
30. **Missing Tool Handling:** Manifest requesting unregistered tool name triggers `UNKNOWN_TOOL_REJECTION`.
31. **Incompatible Data Handling:** Mismatched date ranges trigger `DATA_WINDOW_MISMATCH`.
32. **Claim Inspector Direct vs Interpretation Separation:** Inspector clearly differentiates empirical numbers from analytical narrative.
33. **Case Export Integrity:** Exported JSON parses back into identical `ResearchCase` structure.
34. **Research Trail Sync:** Sealing a case appends a structured entry to the Research Trail.
35. **Ghost Mode Insight Trigger:** Replay match triggers `REPRODUCIBILITY_STABLE` Ghost Mode insight.

21. Failure Modes & Graceful Handling Matrix

Failure Mode	Root Cause	System Response
Corrupted Manifest	Malformed JSON or schema violation	Replay halts at Level 1; emits <code>CORRUPTED_MANIFEST_ERROR</code> .
Missing Tool	Tool removed or unregistered	Replay transitions to <code>FAILED</code> with <code>UNREGISTERED_TOOL_ERROR</code> .
Engine Incompatibility	Breaking engine version bump	Replay transitions to <code>INCOMPATIBLE</code> ; explains version gap.
Dataset Drift	Historical data modified or corrupted	Level 2 audit fails with <code>DATASET_FINGERPRINT_MISMATCH</code> .
Numerical Divergence	Floating-point delta > 10 ⁻⁶	Level 4 audit flags <code>EVIDENCE_MISMATCH</code> with exact deltas.
Replay Timeout	Engine calculation > 60s	Replay halts; records <code>EXECUTION_TIMEOUT_EXCEEDED</code> .
Worker Failure	Web Worker crash or out-of-memory	Gracefully traps worker error; falls back to single-threaded or logs <code>WORKER_FAULT</code> .

22. Proposed Implementation File Architecture

```
src/core/research/audit/  
├─ researchCase.ts           // ResearchCase factory, sealing, immutability guards  
├─ researchManifest.ts      // Manifest builder, canonical serialization, schema validation  
├─ researchCaseStore.ts     // In-memory & localStorage case repository  
├─ replayEngine.ts          // Deterministic re-execution of manifest experiments  
├─ replayVerifier.ts         // 6-level verification audit & mismatch diagnostician  
├─ researchDiff.ts          // Neutral comparative diff engine between cases  
├─ evidenceLineage.ts       // Forward and backward lineage tracing  
├─ claimInspector.ts        // Claim inspection resolver and binding auditor  
├─ auditTimeline.ts         // Append-only audit event ledger  
├─ versionCompatibility.ts   // Engine version matrix and compatibility checker  
├─ caseExport.ts            // Markdown, JSON, and PDF case file generators  
└─ index.ts                 // Barrel export  
  
src/components/workspace/research/audit/  
├─ ResearchCaseView.tsx     // Master case container with tab navigation  
├─ ResearchCaseFile.tsx     // 17-section institutional report view  
├─ ReplayPanel.tsx          // Replay execution launcher & real-time progress  
├─ ReplayVerification.tsx   // 6-level verification audit card & diffs  
├─ ResearchDiffView.tsx     // Side-by-side neutral comparison table  
├─ EvidenceLineage.tsx      // Interactive lineage graph visualization  
├─ ClaimInspector.tsx       // Interactive modal for claim evidence inspection  
├─ AuditTimeline.tsx        // Chronological audit event trail  
└─ CaseExportPanel.tsx      // Multi-format export dialog  
  
tests/  
└─ researchAudit.test.ts    // 35+ automated invariant tests  
  
docs/  
└─ RESEARCH-AUDIT-METHODOLOGY.md // Methodological documentation
```

23. Acceptance Criteria Checklist

Phase 4.1 is accepted only if:

- ☐ `ResearchCase` data model is immutable and serializable
- ☐ Canonical case identity (`BBX-CASE-YYYY-XXXX`) implemented

- ☐ `ResearchManifest` captures complete parameter closure (Data, Strategy, Portfolio, Simulation, Engines, Research, Security)
- ☐ `ResearchReplayEngine` reconstructs and executes planned experiments via `ResearchToolRegistry`
- ☐ 6-level replay verification algorithm implemented (Config → Data → Experiments → Evidence → Claims → Synthesis)
- ☐ Structured mismatch diagnostics isolate exact parameter/evidence/claim divergences
- ☐ `ResearchDiffEngine` compares two cases with strict epistemic neutrality (zero rankings or evaluative judgments)
- ☐ Evidence lineage maps claims bidirectionally to evidence records and data windows
- ☐ Claim inspector displays direct evidence vs. analytical interpretation distinctly
- ☐ 17-section `ResearchCaseFile` implemented with JSON and Markdown export
- ☐ Append-only `AuditTimeline` tracks lifecycle and audit events
- ☐ Version compatibility engine identifies exact vs. incompatible engine versions
- ☐ Zero API keys or secrets persisted in manifests
- ☐ Simulation path ceilings ($\leq 10,000$) and Web Worker boundaries preserved
- ☐ Ghost Mode surfaces `REPRODUCIBILITY_STABLE` and drift events
- ☐ Research Trail receives permanent case logging
- ☐ All 35+ automated tests pass in `tests/researchAudit.test.ts`
- ☐ All existing 306 platform tests continue to pass (zero regressions)
- ☐ TypeScript compiles with 0 errors (`npx tsc --noEmit`)
- ☐ Production bundle builds cleanly (`npm run build`)
- ☐ Non-investment advice disclosures and historical simulation labels preserved